



Research Note: LLD Evaluation Approach

This document explains the research, market gaps, and rationale behind the evaluation and practice system used in **LLD Coach** in simple terms.

1. Why LLD Evaluation Is Difficult

Evaluating Low-Level Design (LLD) is challenging because there is no single "correct" answer. Unlike algorithmic coding problems that produce exact outputs, an LLD problem can have multiple valid object-oriented designs. Because of this, LLD solutions cannot be evaluated with simple pass/fail unit tests alone.

Common Problems in LLD Solutions:

1. **Too much responsibility in one class:** Putting business rules, data management, and coordination into a single large class.
2. **Poor separation of responsibilities:** Missing clear boundaries between different components of the system.
3. **Unnecessary design patterns:** Forcing complex design patterns where simple classes and methods would be cleaner.
4. **Missing edge cases:** Overlooking invalid inputs, boundary limits, and concurrent operations.
5. **Poor domain/entity modeling:** Failing to identify and properly define the core real-world entities required for the problem.

2. Existing Approaches and Key Gaps

When preparing for Low-Level Design interviews or practicing object-oriented design, candidates typically rely on a few common approaches:

1. **Static tutorials / blogs / videos**
 - *Strengths:* Good for learning concepts, design patterns, and seeing worked-out examples.
 - *Gap:* Mostly passive learning with limited personalized feedback on candidate-written solutions.
2. **Coding practice platforms (e.g., LeetCode, HackerRank)**
 - *Strengths:* Good for algorithm and coding problems with automated test suites.
 - *Gap:* LLD design quality (extensibility, clean abstractions, SOLID compliance) is harder to judge with simple pass/fail test cases.
3. **Peer or mock interviews**
 - *Strengths:* Good for real-time discussion, follow-up questions, and human feedback.
 - *Gap:* Feedback is not always consistent, relies on interviewer availability, and cannot easily be done repeatedly on demand.
4. **General AI assistants (e.g., ChatGPT)**
 - *Strengths:* Can provide design suggestions and review code.
 - *Gap:* Without a fixed problem, standard rubric, and structured submission format, feedback can be inconsistent or generic across attempts.

3. Our Product Direction

LLD Coach is designed to support a clear, repeated practice loop:

Choose Problem → Design → Submit → Get Feedback → Review → Try Again

The MVP focuses on:

- A small set of **curated LLD problems** covering essential interview scenarios.
- A **structured 4-section submission workspace** to guide the candidate's design thinking.
- A **hybrid evaluation pipeline** combining fast rule-based checks with AI-driven qualitative feedback and safe fallback.
- **Attempt history and version tracking** so candidates can review past attempts, refine their designs, and track improvement over time.

4. Why the 4-Part Submission Format

Rather than requiring learners to build complex visual diagrams, LLD Coach uses a structured 4-part text submission format:

1. **Design Explanation:** Overview of high-level architecture, approach, and how system components interact.
2. **Classes & Responsibilities:** Definitions of core entities, interfaces, relationships, attributes, and methods.
3. **Code / Pseudocode:** Concrete implementation of primary classes and core business logic.
4. **Trade-offs & Assumptions:** Discussion of edge cases, scalability limits, concurrency considerations, and alternative design decisions.

Why this format was chosen:

- It captures clear evidence of object-oriented design thinking without the overhead of building and managing a visual UML/diagram editor.
- It mirrors what interviewers expect during real LLD interviews (explaining architecture, defining models, writing code, and justifying trade-offs).
- It provides structured sections that the evaluation pipeline can inspect consistently.

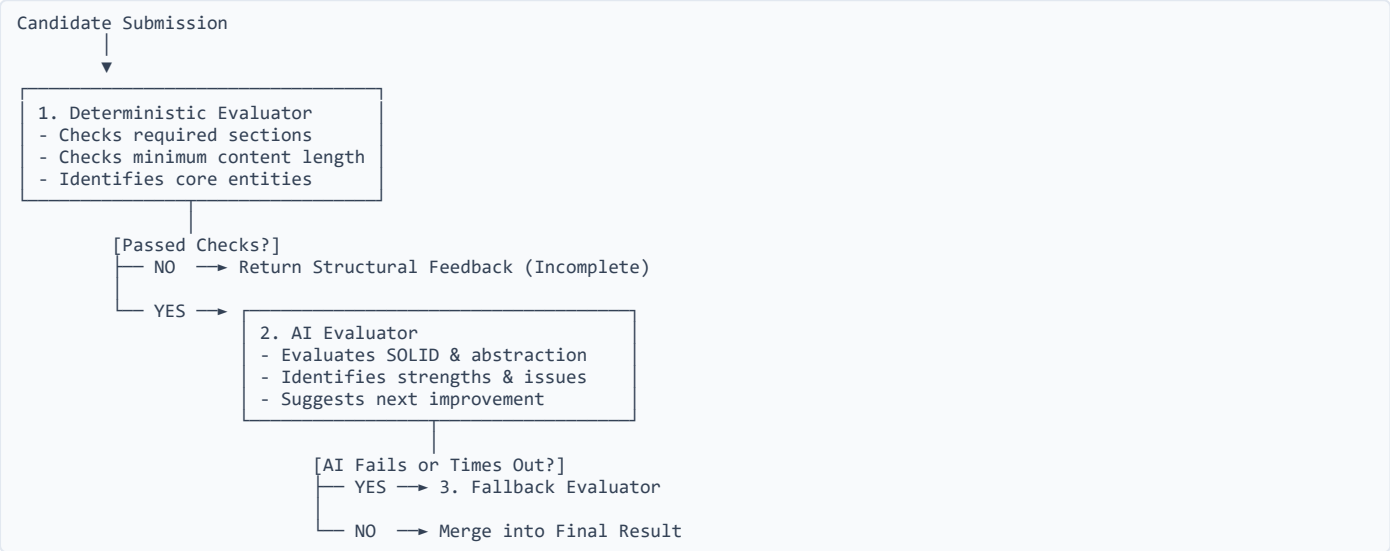
5. Evaluation Rubric

To provide structured and fair feedback, LLD Coach uses a **100-point scoring model** divided across 4 main areas (25 points each):

Category	Points	Description
SOLID Principles	25	Checks for Single Responsibility (SRP), Open/Closed Principle (OCP), and clean interface separation.
Class Design & Abstraction	25	Checks whether the expected core entities, relationships, attributes, and methods are modeled clearly.
Extensibility & Patterns	25	Checks if appropriate design patterns (such as Strategy, State, or Factory) are used naturally without over-engineering.
Edge Cases & Concurrency	25	Checks whether edge cases, error conditions, boundary constraints, and concurrency trade-offs are considered.

6. Hybrid Evaluation

The evaluation engine uses a hybrid approach to evaluate submissions:



- **Deterministic Evaluator:** Runs fast rule-based checks on structure, minimum section content, and mentions of core entities.
- **AI Evaluator:** Analyzes design quality and provides qualitative, explainable feedback.
- **Fallback Evaluator:** Generates rule-based feedback if the AI provider is unavailable, times out, or fails.
- **EvaluationPipeline:** Coordinates the entire evaluation flow by running deterministic checks first, then invoking the AI evaluator with fallback.

7. Why Hybrid Evaluation

1. **Consistent Basic Checks:** Deterministic checks quickly verify that required sections and core entities are present.
2. **Explainable Design Feedback:** The AI evaluator can read class structures and explain *why* a design works well and *how* to refactor it.
3. **Safe Fallback:** The fallback evaluator helps the evaluation flow continue with basic feedback when AI evaluation is unavailable or fails.
4. **Actionable Learning:** Instead of just a single score, the learner receives category scores, strengths, categorized issues with severity tags, and a clear next step to improve their design.